ROS 2 · ADAS · AUTOMATIC EMERGENCY BRAKING

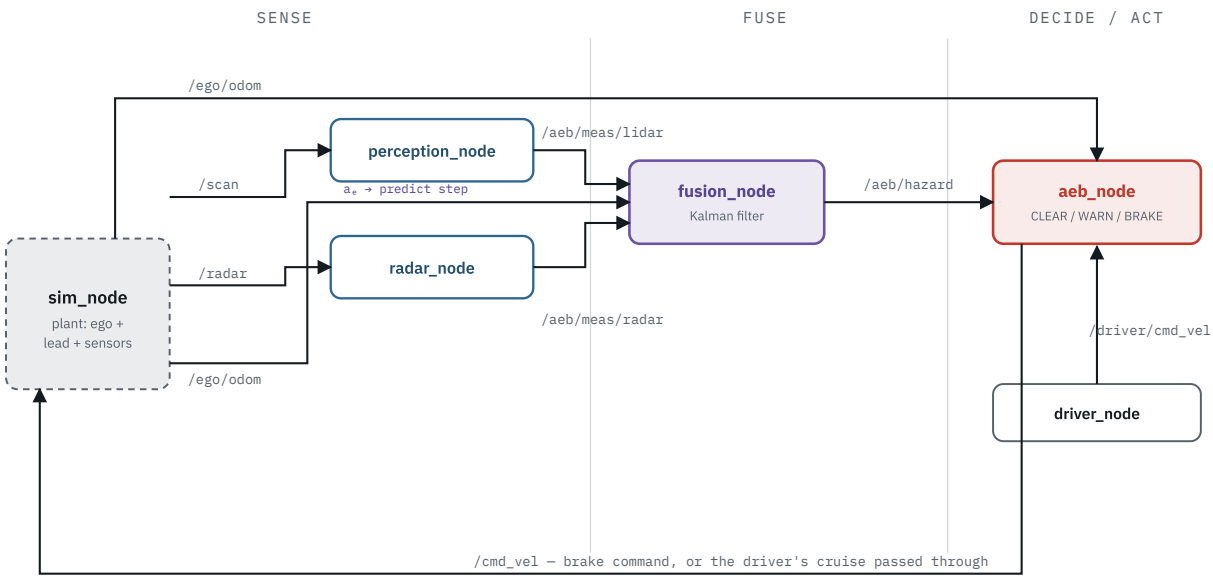
How the AEB system works — one diagram at a time

Six pictures that together explain the whole pipeline: what each node does, why two sensors are fused, how the Kalman filter runs, and how the braking decision is made. Each has a line you can say out loud.

01

The pipeline

Data flows one way: sensors → fusion → decision → actuation, then the command loops back to the vehicle.



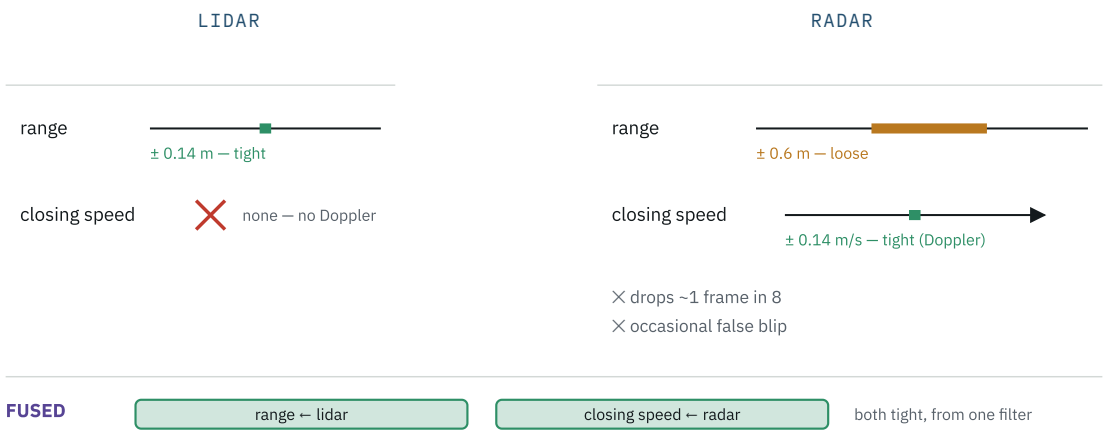
The loop. `sim_node` is the stand-in for the real vehicle and its sensors. Everything else is the stack you'd keep if you swapped the sim for real drivers. `/ego/odom` feeds `fusion_node` (as a control input — diagram 3) and `aeb_node` (to know when the car has stopped).

> **Say:** "Four processing nodes with single responsibilities — two turn a sensor into a measurement, one fuses, one decides. No node sees ground truth; each only gets the topic above it."

02

Why fuse two sensors

Each sensor is good at one thing and bad at the other. Together they cover both — that's the whole reason the fusion node exists.

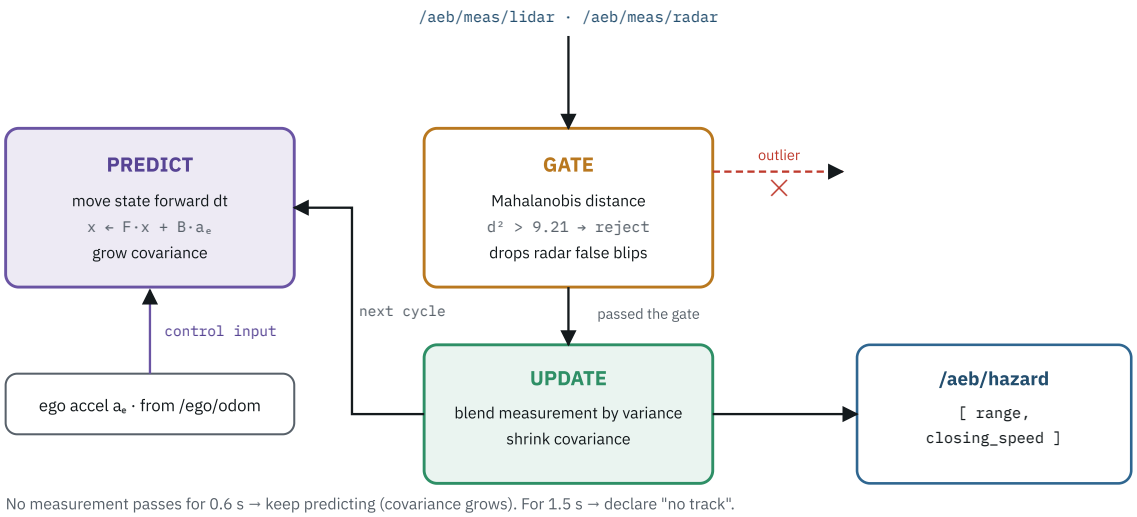


Complementary. The Kalman filter weights each measurement by the variance the sensor reports, so it automatically takes range from the lidar and closing speed from the radar — no hand-written "if radar then..." switch.

> **Say:** "Lidar knows where, radar knows how fast. Neither alone is enough to decide a brake, so I fuse them — and the fusion also has to survive the radar dropping out and lying."

The Kalman filter loop

Runs every cycle. Predict the state forward, check each measurement is plausible, blend in the ones that pass.

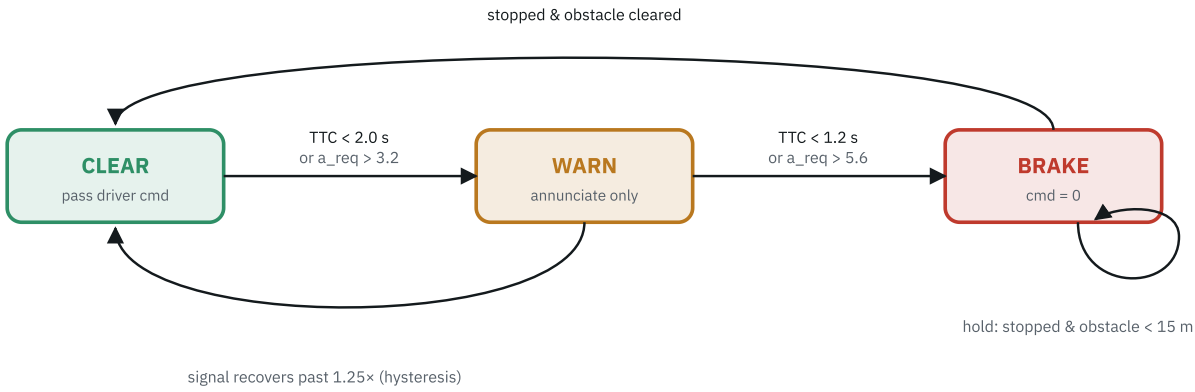


Predict–gate–update. The state is `[range, closing_speed]`. Feeding the ego's own deceleration into PREDICT is what stops the filter fighting a hard brake — without it, predictions diverge from reality and the gate rejects the correct measurements.

> **Say:** "Constant-closing-speed model, but I feed in the ego's own acceleration as a known input so the filter tracks through braking. The gate throws out radar false alarms; if everything stops passing, it stops trusting the track."

The braking state machine

Three states. WARN only annunciates; BRAKE commands full braking and holds it.

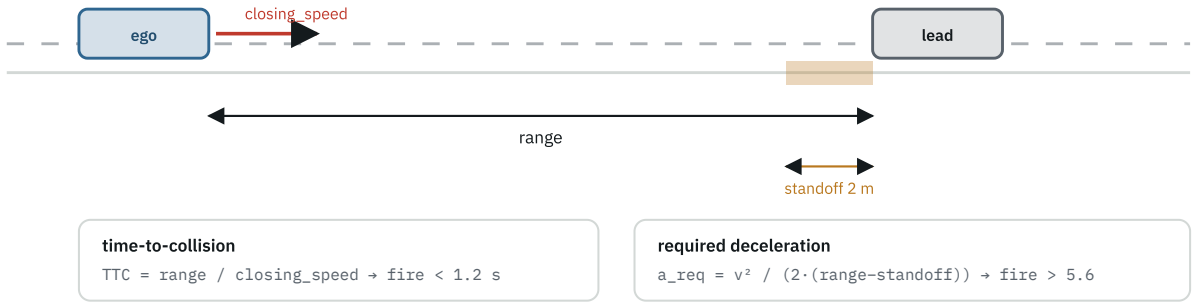


Latch and hold. Once in BRAKE it stays until the car is stopped, then keeps `cmd = 0` while the obstacle is still close — production "AEB stop and hold". The hysteresis on the WARN→CLEAR edge stops the state flickering on a noisy signal sitting on the threshold.

> **Say:** "Two thresholds to escalate, a relaxed one to de-escalate so it doesn't chatter, and BRAKE latches to a full stop and holds — it won't release mid-event."

The two brake gates

Both are computed from the same fused `range` and `closing_speed`. Either one crossing its threshold triggers the brake.

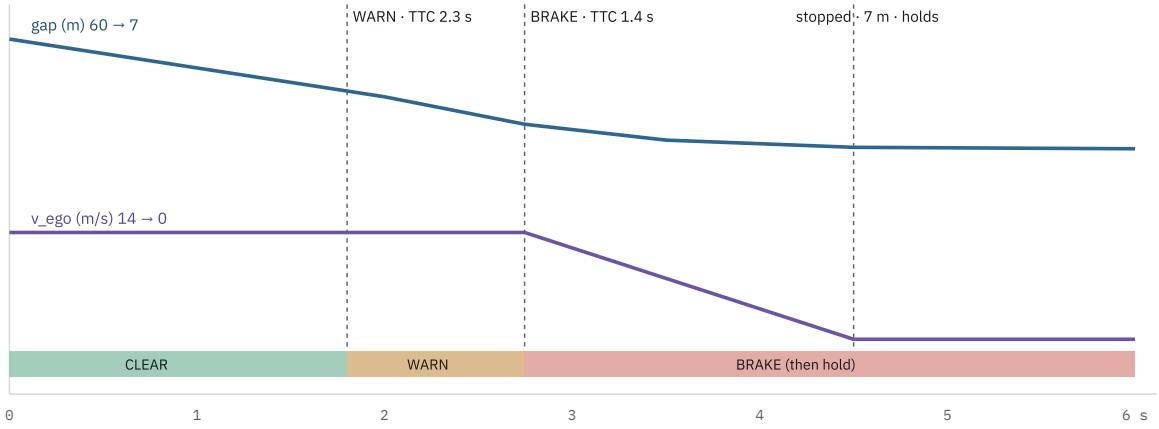


Two views of the same danger. TTC asks "how long until impact?". Required deceleration asks "can I still stop in the space left?" — it catches high-speed cases where TTC reacts too late. `5.6 = 0.7 × 8 m/s²`, i.e. 70% of the car's braking capability.

> **Say:** "Defense in depth — TTC is intuitive but blind to whether stopping is still possible; the deceleration gate covers that. Either one fires the brake."

One run, start to stop

`stationary_lead`: ego at 14 m/s, stopped car 60 m ahead. Numbers are from an actual run.



The whole event is ~3 seconds. WARN at t≈1.8 s gives ~1 s of annunciation before BRAKE at t≈2.75 s; full braking pulls 14 m/s to 0 in about 1.7 s, stopping ~7 m short of the lead, then holds.

> **Say:** "It escalates CLEAR → WARN → BRAKE as time-to-collision drops, brakes to a full stop with margin to spare, and holds. Disable the safety node and the same run ends in a collision."

ROS 2 Â· ADAS Â· SYSTEM ENGINEERING

Automatic Emergency Braking, built to talk about

A field guide to the AEB demo project â€” what it does, why each design decision was made, and the questions to expect. Read it until you can redraw the node graph, re-derive the two brake gates, and sketch the Kalman filter on a whiteboard.

> **Ground rule:** you built this, you ran it, you can explain every line. That makes *"I implemented an AEB pipeline in ROS 2 with lidarâ€“radar fusion"* a true statement. Don't stretch it into production or on-vehicle experience â€” the value is that you understand a real safety function end to end, including its limits.

01 The 30-second version

When they say *"tell me about a project"*:

> **"I built an Automatic Emergency Braking pipeline in ROS 2. A simulator publishes a synthetic lidar and radar; a perception and a radar node each turn their sensor into a measurement with a variance; a fusion node runs a Kalman filter that combines them, gates outliers, and coasts through radar dropouts; and a safety node runs a CLEAR / WARN / BRAKE state machine with two independent braking criteria and a stop-and-hold. The fusion and braking maths are pure, unit-tested modules with no ROS dependency. It stops the ego from 14 m/s behind a stopped car, and drives into it when I disable the safety node."**

02 Architecture

Six nodes, four custom messages, two plain-Python logic modules. The **sense â†’ fuse â†’ decide â†’ act** split is the point â€” each node does one job and is testable on its own.

NODE	CONSUMES	PRODUCES	JOB
sim_node	/cmd_vel	/scan, /radar, /ego/odom	Longitudinal model of ego + lead; synthetic lidar and radar
perception_node	/scan	/aeb/meas/lidar	Accurate in-path range (no velocity â€” that's the radar's job)
radar_node	/radar	/aeb/meas/radar	Validity-gates returns; Doppler closing speed (tight), range (loose)
fusion_node	/aeb/meas/lidar, /aeb/meas/radar	/aeb/hazard	Kalman filter; outlier gating; coasting through dropouts
aeb_node	/aeb/hazard, /ego/odom, /driver/cmd_vel	/cmd_vel, /aeb/status	State machine; overrides the driver command with full braking
driver_node	â€”	/driver/cmd_vel	Stand-in driver holding a cruise speed for AEB to supervise

No node uses ground truth. Perception and radar see only their own noisy measurements; `aeb_node` sees only the fused `Hazard`. Swapping the simulator for real sensor drivers is a launch-file change.

03 Sensor fusion

Neither sensor alone is enough: lidar has an accurate range but a poor velocity; radar has an accurate closing speed but a noisy range, and it drops out and throws false alarms. A Kalman filter turns the pair into one clean track.

```
x = [ range , closing_speed ]   d(closing_speed)/dt = a_ego âˆ’ a_lead
predict: x â† FÂ·x + BÂ·a_ego   F = [[1, âˆ’dt], [0, 1]]
```

The ego's own deceleration (from `/ego/odom`) is a **known control input** to the predict step â€” only the lead's acceleration is unknown, and it becomes the process noise `Ïƒ_a`. Drop that `BÂ·a_ego` term and a constant-velocity model fights the braking AEB just commanded: the prediction diverges, the gate rejects the correct measurements, and the track falls apart. That was a real bug in this project; the fix was one term.

SENSOR	MEASURES	RANGE VAR	RATE VAR
lidar	range only	0.02 mÂ²	â€”
radar	range + rate	0.36 mÂ²	0.02 (m/s)Â²

The Kalman gain does the weighting on its own: **range follows the lidar, closing speed follows the radar**, with no hand-coded switch. Two more pieces:

- **Mahalanobis gate.** Each measurement is scored against the prediction; `dÂ² > 9.21` (Ï‡Â², 2-dof, 99 %) is rejected â€” that drops radar false alarms with no separate filter. If a joint range+rate measurement fails, the safety-critical range is retried on its own.
- **Coasting.** No accepted measurement for `coast_timeout` â†’ predict on the model with inflating covariance. Past `drop_timeout` â†’ invalidate the track. A blind sensor stack should not brake on a guess.

04 The two brake gates

Either criterion alone can request a brake â€” defense in depth. They fail toward braking, not away from it.

Time-to-collision gate

```
TTC = range / closing_speed   â†’   brake if TTC < 1.2 s
```

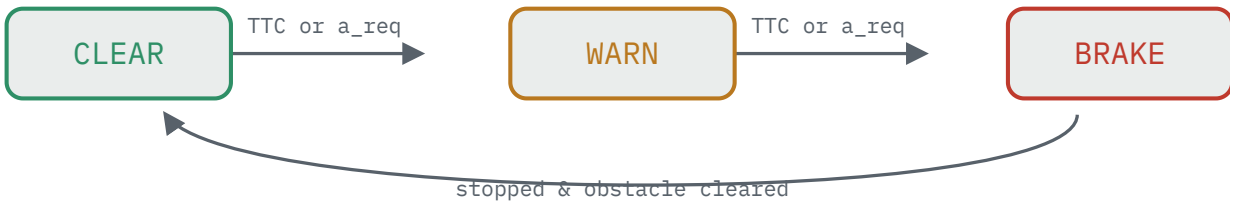
Cheap and intuitive. Weakness: it says nothing about whether you *can* still stop â€” at high speed, by the time TTC is low the situation is already lost.

Required-deceleration gate

```
a_req = closing_speedÂ² / (2 Â· (range â^’ standoff))
brake if   a_req > 0.7 Â· a_max   (a_max = 8 m/sÂ²)
```

From vÂ² = 2Â·aÂ·d. Trips when comfortable braking is no longer enough to stop `standoff` metres short. Catches the high-speed cases the TTC gate reacts to too late. `standoff` â‰ˆ 2 m keeps the stopping point clear of the obstacle.

05 The state machine



- **WARN** is advisory only â€” no actuation. It exists to drive a forward-collision-warning HMI.
- **BRAKE** commands full braking, then latches until the vehicle is stopped, then *holds* the brake while an obstacle stays within `hold_distance`. Real "AEB stop and hold" behaves this way; it also stops the ego creeping forward again once the driver's foot is still on the gas.

06 Why hysteresis

The `a_req` estimate is noisy â€” closing speed is a filtered finite difference with a ~0.5 m/s noise floor. Sitting exactly on a threshold, the raw logic flips state many times a second.

Fix: **entering** a more-severe state uses the nominal threshold; **leaving** it requires the signal to recover past a 1.25Ã— margin. One known residual: a single brief `CLEARâ†'WARNâ†'CLEAR` flap can still happen on first entry, because hysteresis only damps the exit. WARN drives nothing, so it's cosmetic â€” and being able to name it is a better answer than pretending it's perfect.

07 ROS 2 concepts it exercises

Everything here is something you can be asked to point at in the code:

`publishers / subscribers` `timers` `multi-topic callback` `parameters + YAML` `launch files` `launch args & conditions` `4Ã— custom .msg` `ament_python` `ament_cmake (interfaces)` `tf2 broadcast` `QoS defaults` `colcon build / test`

`RViz + MarkerArray`

One war story worth telling: the pipeline first ran silently doing nothing. Root cause was a non-finite value (`inf`) written into a `float32` message field, which killed the perception callback. Fix was a finite sentinel plus a guarded callback that logs instead of dying quietly. Good example of debugging a distributed graph where "no error" doesn't mean "working".

08 Known limitations

Say these before you're asked â€” knowing the edges of your own system is the signal.

- Purely longitudinal: no steering, no lateral path, one obstacle â€” so the "fusion" is a single track with fixed association, not JPDA / MHT.
- Sensors are synthetic. Lidar noise is Gaussian with no clutter or occlusion; radar has dropouts and false alarms but no multipath, angle, or RCS model.
- No track confirmation â€” a radar false alarm as the *first* return can delay track acquisition by up to `drop_timeout` (~1.5 s).
- First-order vehicle model: no actuator lag, no brake-pressure ramp, no tyre model.
- Fixed thresholds. Production AEB schedules them on speed, estimated road friction, and driver-attention state.
- Not ISO 26262 anything: no redundancy, no fault handling, no plausibility checks on the sensor stream itself.

09 Questions to expect

Q Why two braking criteria instead of one?

A They cover different failure modes. TTC is blind to whether stopping is still physically possible; the deceleration gate is blind to slow-closing cases that are still imminent because they're very near. Requiring either to trip is a cheap safety margin.

Q Why fuse the sensors instead of picking the better one?

A They're better at different things â€” lidar range, radar velocity â€” and each has failure modes the other covers. The Kalman filter weights each measurement by its variance, so I get the best of both plus a track that survives radar dropouts by coasting on the motion model.

Q How does the filter reject a radar false alarm?

A A Mahalanobis distance check: score the measurement against the predicted state and its covariance, reject if it exceeds a chi-square threshold. A spurious return 30 m off the track is hundreds of sigma out, so it never gets applied.

Q A constant-velocity filter can't track hard braking. How do you handle it?

A The ego's own acceleration is a known control input â€” I feed it into the predict step from the odometry, so the filter doesn't fight the braking AEB commanded. Only the *lead's* acceleration is left as process noise. The lead can still surprise it; for that I'd move to a constant-acceleration or IMM model.

Q Your controller chatters near the threshold. What do you do?

A Hysteresis â€” asymmetric enter/exit thresholds â€” plus latching the brake state until the vehicle has stopped. If it still chattered I'd add a debounce counter or filter the decision input harder, trading reaction time for stability.

Q How did you test it?

A Both logic modules are pure functions with no ROS imports. `safety.py` has a unit test per transition; `fusion.py` has tests for filter convergence, range-only vs rate-only updates, outlier gating, and complementary weighting. Integration check: run the scenarios, assert no collision with AEB on and collision with it off.

Q What would you add next?

A A camera measurement as a third input (class + rough range); an Adaptive Cruise Control mode that follows at a time-gap and hands off to AEB; replaying a recorded `ros2 bag` in a regression test.

Q Why ROS 2 and not just a script?

A The node boundaries force clean interfaces â€” perception can't reach into the controller's state. I get introspection for free (`ros2 topic echo`, `rqt_graph`, bags), and swapping the simulator for real sensor drivers is a launch-file change, not a rewrite.

`aeb_ros2_demo Â· ROS 2 Humble Â· sim â†’ lidar + radar â†’ KF fusion â†’ AEB`